



DeliveryHub

ADMIN PORTAL

Secure Shipment Management & Authentication System

A Flask-based platform for secure application onboarding,
shipment lifecycle management, and audit-ready authentication logging.

Flask

MySQL

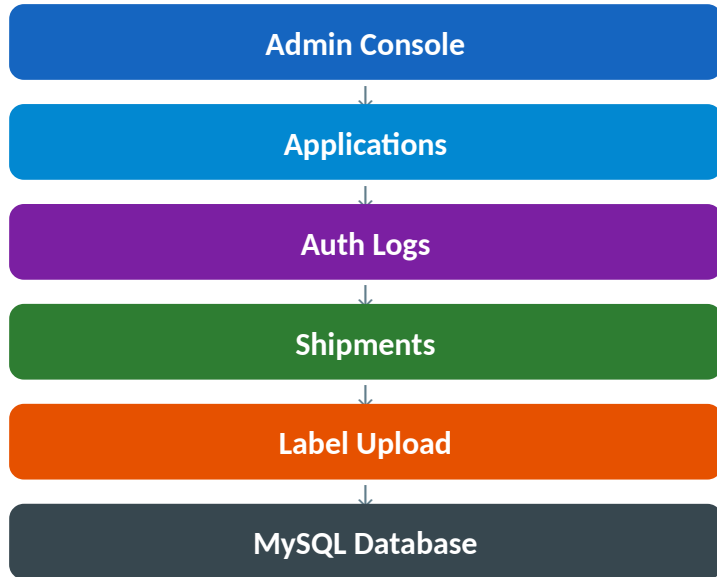
Python

REST API

System Architecture

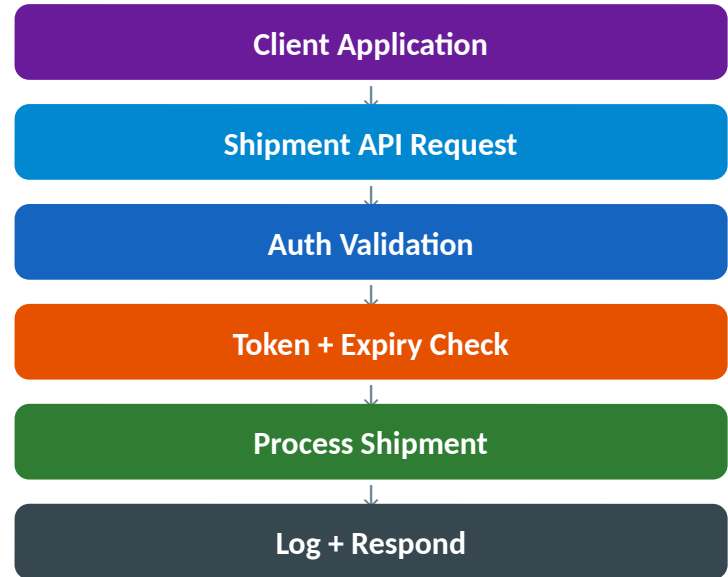
How DeliveryHub components interact

Admin Portal Flow



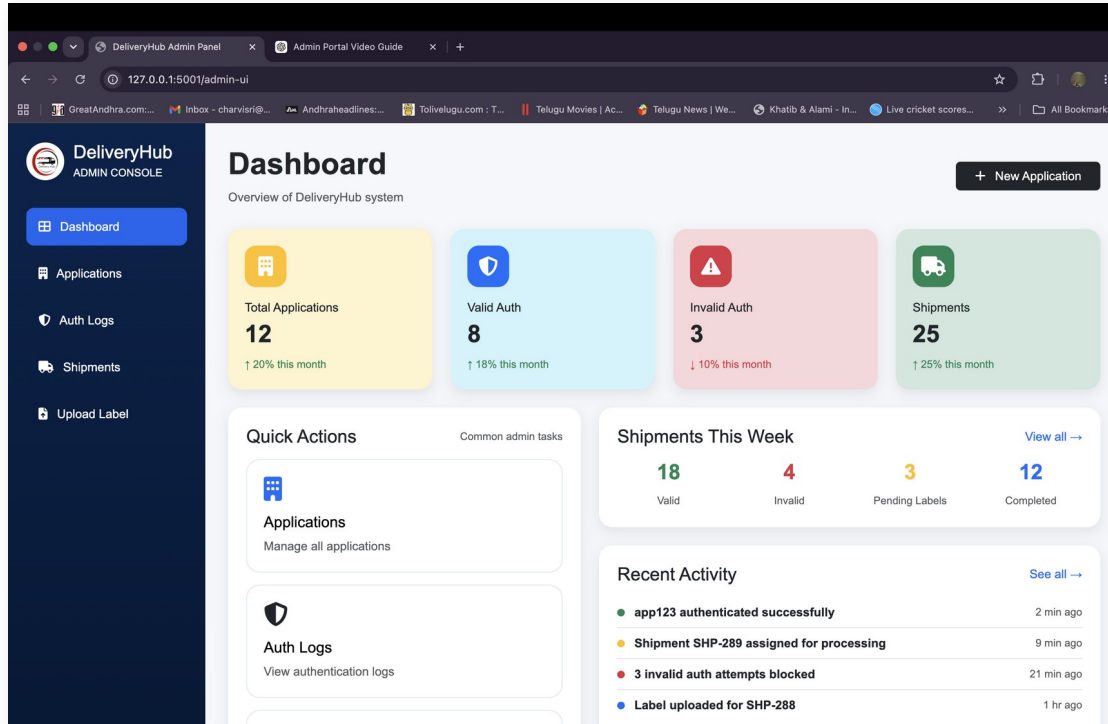
MySQL
DB

External Client Flow



Admin Dashboard

System-wide overview at a glance



Total Applications

All onboarded client apps



Valid Auth

Successful auth attempts



Invalid Auth

Failed / blocked requests

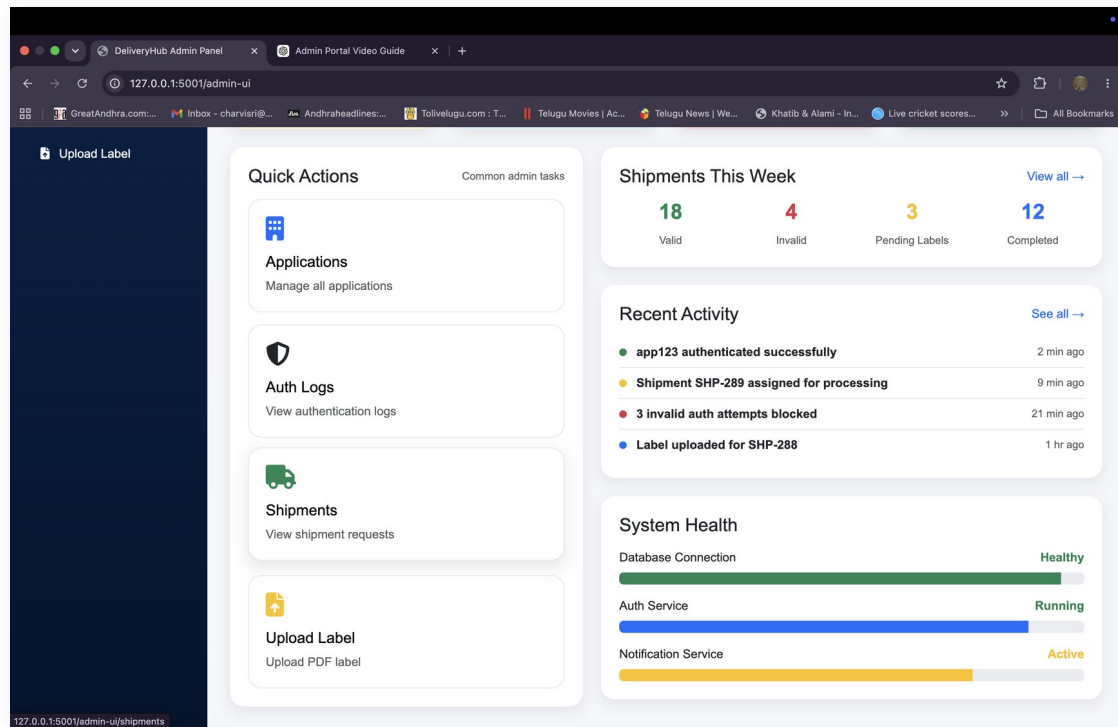


Shipments

Total requests processed

Dashboard — Quick Actions & Health

Shortcuts and live service monitoring



System Health

Database Connection

Healthy

Auth Service

Running

Notification Service

Active

Applications Module

Manage client application onboarding, status, and expiry

DeliveryHub
ADMIN CONSOLE

Applications
Manage application access, expiry date and active status.

+ New Application

Total Applications
15

Active Applications
14

Inactive Applications
1

Expiry Managed

Registered Applications

APPLICATION ID	NAME	EMAIL	EXPIRY DATE	STATUS	ACTIONS
80ccca89-fa88-474a-9dc4-116e5e744d76	Integration Test App	test@example.com	2026-09-03	Active	Already Active 03/09/2026 ☐ <button>Update</button>
7b5e85b5-e08b-4102-b4b9-ft4592afdabd	Delivery_hub	charvisri2006@gmail.com	2026-06-11	Active	Already Active 11/06/2026 ☐ <button>Update</button>
45e63d52-d201-45d8-8a01-45d43474d38f	Delivery_hub	charvisri@gmail.com	2026-08-31	Active	Already Active 31/08/2026 ☐ <button>Update</button>
89502739-a5ce-41de-9408-2b815dcf60e36	Delivery_hub	charvisri@gmail.com	2026-08-31	Active	Already Active 31/08/2026 ☐ <button>Update</button>
a1c09b76-dcf2-475d-aac9-cdc539bb774e	Delivery_hub	charvisri@gmail.com	2026-08-31	Active	Already Active 31/08/2026 ☐ <button>Update</button>

Database Table: applications

application_id

UUID — used for auth

application_token

Hashed (never plaintext)

application_name

Client app name

user_email

Credential delivery

expiry_date

Validity period

is_active

Enable / disable access

created_at

Audit timestamp

Create New Application

Onboarding new client applications with secure credential generation

The screenshot shows a web browser window with the URL `127.0.0.1:5001/admin-ui/create-application`. The page is titled "Create New Application" and has a sub-header "Onboard a new client and send application credentials by email." The main content area features a form titled "Application Onboarding" with the subtitle "Admin creates application access for backend integration." The form has two input fields: "Application Name" with the example text "Example: Sateesh Logistics" and "User Email" with the example text "sateesh@example.com". Below the fields is a dark blue button with the text "+ Create Application". The left sidebar of the admin console contains links for "Dashboard", "New Application", "Applications", "Auth Logs", "Shipments", and "Upload Label".

What happens on submit:

1

Generate UUID as Application ID

2

Generate secure random token

3

Hash token before storing

4

Save application to MySQL

5

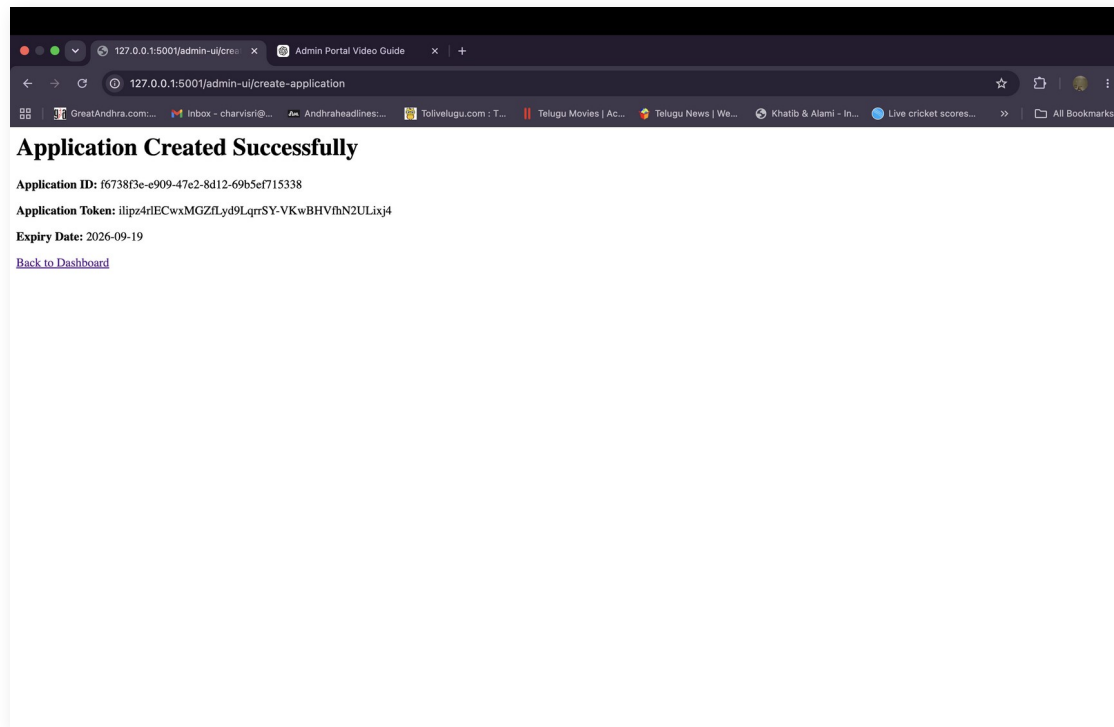
Set default 90-day expiry

6

Trigger Notification Service

Credential Generation

Token shown once — only its hash is stored in the database



One-Time Display



Token visible only on this screen



Hash-Only Storage



Database never stores plaintext token



Auto Expiry Set



90-day default validity period



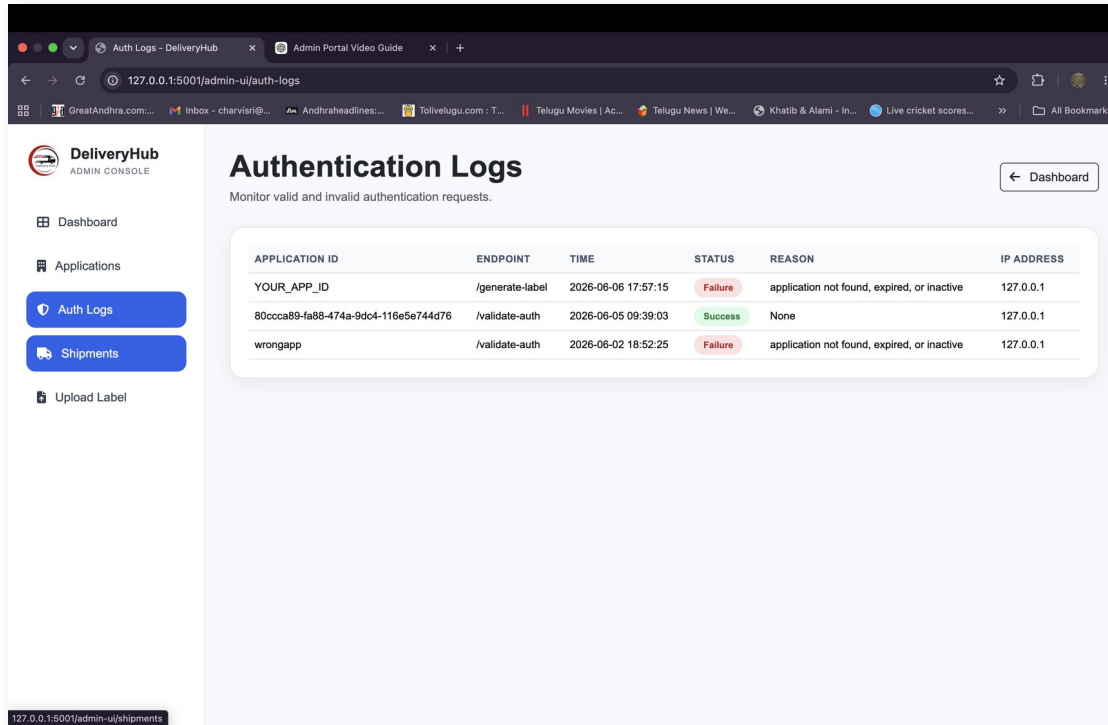
Email Delivered



Credentials sent automatically to owner

Notification Service

Automated email delivery of application credentials



Email Contains:

✓ Application ID

✓ Application Token

✓ Expiry Date

Implemented as a separate notification service integrated with Flask — eliminates manual credential sharing.

Authentication Logs

Full audit trail of every API authentication attempt

The screenshot displays the DeliveryHub Admin Console interface. The left sidebar contains navigation links: Dashboard, Applications, Auth Logs, Shipments (highlighted), and Upload Label. The main content area is titled 'Shipment Requests' with a subtitle 'View sender, receiver, route, validation and label status.' It includes four summary cards: Total Shipments (7), Valid Shipments (7), Invalid Shipments (0), and Labels Uploaded (5). Below these is a table titled 'All Shipments' with a search bar and columns for Shipment ID, Request ID, Sender, Receiver, Route, Service, Validation, Label Status, State, and Action. The table lists five shipment records.

SHIPMENT ID	REQUEST ID	SENDER	RECEIVER	ROUTE	SERVICE	VALIDATION	LABEL STATUS	STATE	ACTION
3	3989a07a-14c6-489c-b3f8-d841c0514855	John Smith johnsmith@gmail.com	Charvi s charvisr2006@gmail.com	USA → India	Express International	Valid	Pending	initiated	<button>Upload Label</button>
2	REQ-SHP-001	John Doe None	Alice Doe None	India → India	Express	Valid	Pending	initiated	<button>Upload Label</button>
1	REQ001	John Doe None	Alice Doe None	India → India	Express	Valid	Label Uploaded	label_uploaded	<button>Done</button>
1	REQ001	John Doe None	Alice Doe None	India → India	Express	Valid	Label Uploaded	label_uploaded	<button>Done</button>
1	REQ001	John Doe None	Alice Doe None	India → India	Express	Valid	Label Uploaded	label_uploaded	<button>Done</button>

Log Fields

Application ID Which app made the request

Endpoint API path accessed

Timestamp Exact time of attempt

Status Success or Failure

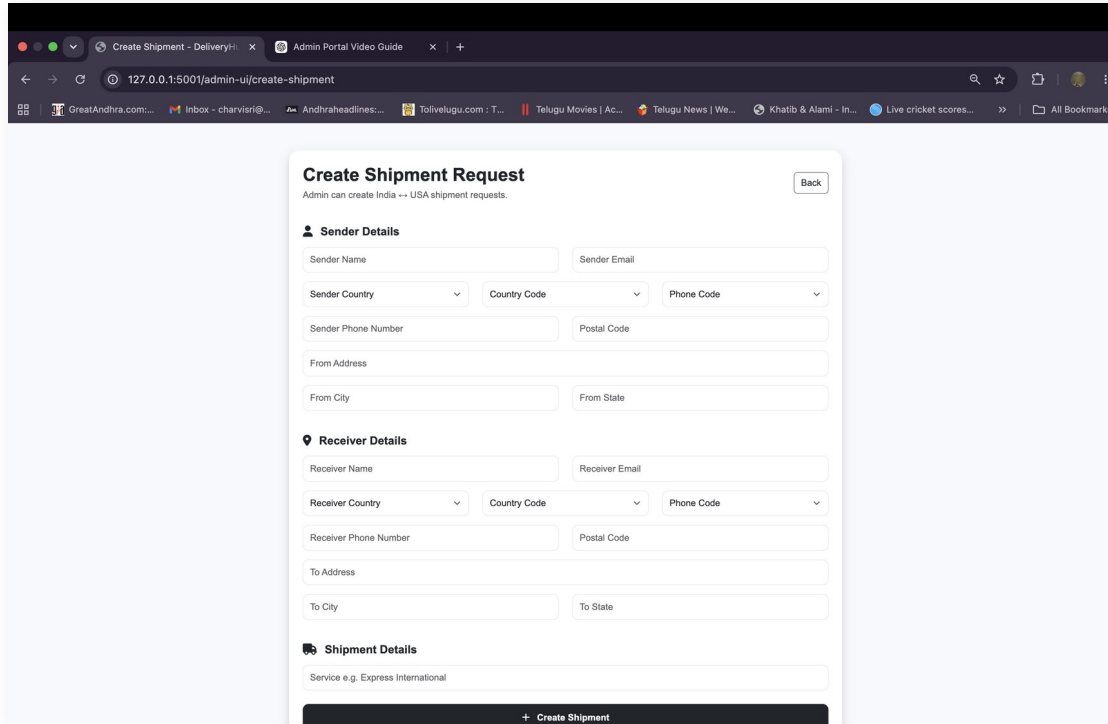
Reason Failure explanation

IP Address Origin of request

Failure Reasons: App not found • Token mismatch • Expired • Inactive

Shipment Requests

Track all shipments, validation status, label status, and lifecycle state



The screenshot shows a web browser window with the address bar displaying '127.0.0.1:5001/admin-u/create-shipment'. The page title is 'Create Shipment Request'. Below the title, there is a note: 'Admin can create India ↔ USA shipment requests.' and a 'Back' button. The form is divided into three main sections: 'Sender Details', 'Receiver Details', and 'Shipment Details'. Each section contains various input fields for personal and contact information, including names, emails, countries, phone codes, phone numbers, postal codes, addresses, cities, and states. At the bottom of the form, there is a 'Shipment Details' section with a field for 'Service e.g. Express International'. A '+ Create Shipment' button is located at the bottom right of the form.

Create Shipment Request Back

Admin can create India ↔ USA shipment requests.

Sender Details

Sender Name Sender Email

Sender Country Country Code Phone Code

Sender Phone Number Postal Code

From Address

From City From State

Receiver Details

Receiver Name Receiver Email

Receiver Country Country Code Phone Code

Receiver Phone Number Postal Code

To Address

To City To State

Shipment Details

Service e.g. Express International

[+ Create Shipment](#)

Shipment States

Valid

Passed all validations

Invalid

Failed validation rules

Pending
Label

Awaiting label upload

Completed

Label uploaded & sent

Create Shipment Request

Full sender/receiver form with country validation and DB multi-table write

The screenshot shows a web browser window with the URL `127.0.0.1:5001/admin-u/upload-label`. The page has a blue header with a PDF icon and the title "Upload Shipment Label". Below the header, it says "Upload a PDF label and send it automatically to the receiver." The form includes a "Shipment ID" field with an example value of "1" and a note that the receiver email will be fetched automatically. There is a "Select PDF Label" section with a cloud upload icon and a note that only PDF files are allowed. Below this is a "Choose File" button and a "No file chosen" status. At the bottom, there is a dark blue "Upload and Email Label" button and a link to "Back to Shipments".

Backend creates records in:

customers

Sender & receiver records

addresses

From/To address details

shipments

Core shipment record

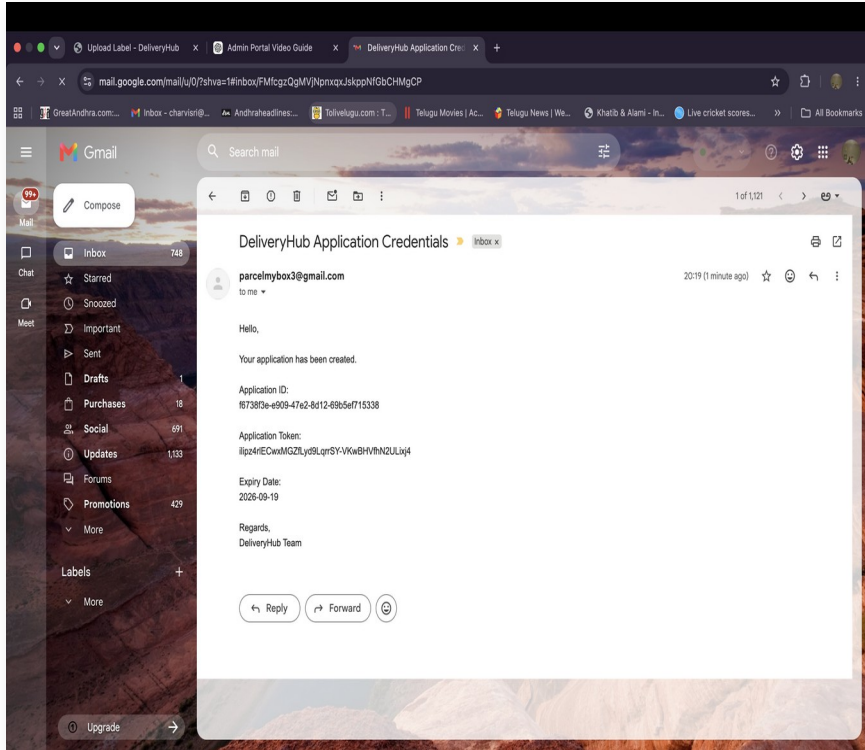
shipment_tracking

Tracking entry created

Validates input → country → address → customer → shipment → tracking

Upload Shipment Label

PDF label upload with automatic email delivery to receiver



Upload Process:

1

Check shipment exists in database

2

Validate file has .pdf extension

3

Save file securely on server

4

Update shipment label status

5

Email label PDF to receiver

Security Features

Security was a major focus of this project



Token Hashing

Tokens stored only as bcrypt hashes —
plaintext never persists in DB



Expiry Validation

Every auth request checks expiry date —
expired apps auto-fail



Active Status Check

Disabled applications cannot access any API
endpoint



Authentication Logging

Every attempt recorded with timestamp,
reason, and IP address



PDF Validation

Label upload strictly validates .pdf extension
before storing



One-Time Token Display

Credentials shown once — never retrievable
again after creation

Future Work

Next steps to take DeliveryHub to production



Dockerization

Containerize all services with Docker Compose



Production Deployment

Deploy on cloud infrastructure



Role-Based Access Control

Fine-grained permissions per admin user



Lifecycle Automation

Automate shipment state transitions



Analytics Dashboard

Usage metrics and trend reporting



Microservices

Full containerized microservice deployment



Thank You

DeliveryHub

Secure application onboarding · API authentication · Shipment management · Label delivery

Built with Flask · MySQL · Python · Email Notification Service